

# CSE 221: Algorithms

## Chapter 6 — Graph Traversal

Prepared by Ariyan Hossain [AYO]

### 1 Graph Traversal

The most fundamental graph problem is to visit every edge and vertex in a graph in a systematic way. Mazes are naturally represented by graphs, where each graph vertex denotes a junction of the maze, and each graph edge denotes a hallway in the maze. Thus, any graph travel algorithm must be powerful enough to get us out of an arbitrary maze. For *efficiency*, we must make sure we don't get trapped in the maze and visit the same place repeatedly. For *correctness*, we must do the traversal in a systematic way to guarantee that we get out of the maze. Our search must take us through every edge and vertex in the graph.

Given a graph  $G = (V, E)$  and a starting vertex  $s$ , graph traversal systematically visits vertices and edges to explore the graph. The two most common traversal strategies are:

- **Breadth-First Search (BFS)**: Explores level by level (in waves).
- **Depth-First Search (DFS)**: Explores as deep as possible before backtracking.

### 2 Breadth-First Search (BFS)

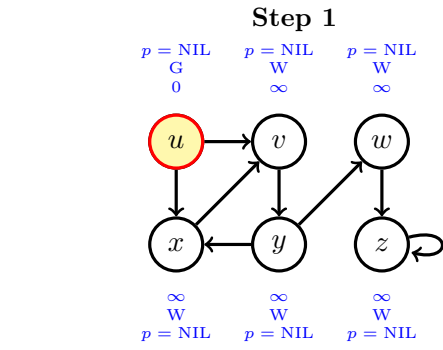
Breadth-first search (BFS) search systematically explores the edges of  $G$  to discover every vertex that is reachable from  $s$ . It computes the distance from  $s$  to each reachable vertex, where the distance to a vertex  $v$  equals the **smallest number of edges** needed to go from  $s$  to  $v$ .

BFS is named for how it explores a graph by expanding outward uniformly from the starting point. You can imagine it as discovering new nodes in waves that move outward from the source node. Starting from the source, BFS first finds all nodes directly connected to it. Then, it moves to the nodes that are 2 steps away, then those 3 steps away, and continues this process until all reachable nodes are found.

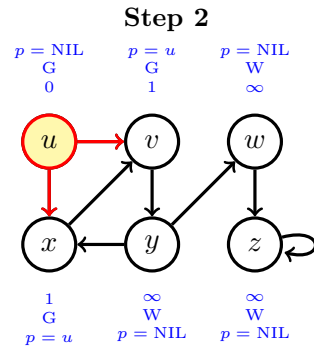
To keep track of nodes, BFS uses a queue that follows **First In First Out (FIFO)**.

BFS also keeps track of:

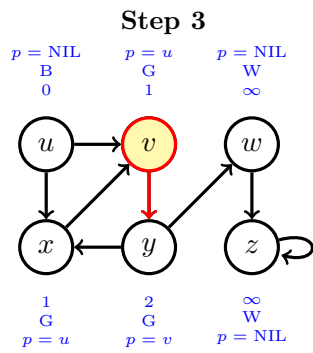
- $\text{color}[v]$  : state of each vertex
  - White (W): undiscovered
  - Gray (G): discovered but not finished processing
  - Black (B): processed
- $\pi[v]$  or  $p[v]$  : predecessor (parent) of  $v$
- $d[v]$  : distance (number of hops) from the source  $s$

2.1 Simulation (BFS from source  $u$ )

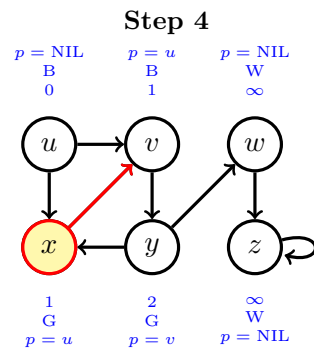
Queue:  $[u]$   
 Current: none yet  
 Neighbors: none yet



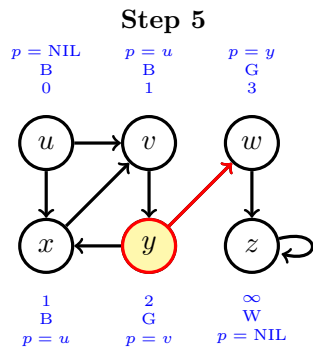
Queue:  $[v, x]$   
 Current:  $u$   
 Neighbors:  $v, x$



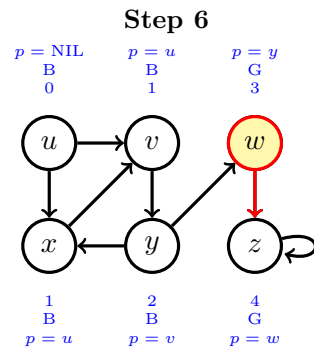
Queue:  $[x, y]$   
 Current:  $v$   
 Neighbors:  $y$



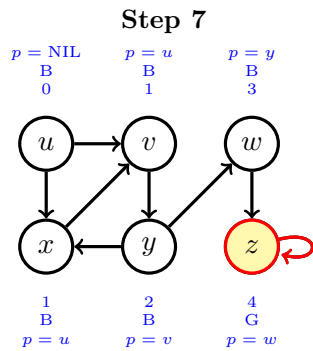
Queue:  $[y]$   
 Current:  $x$   
 Neighbors:  $v$



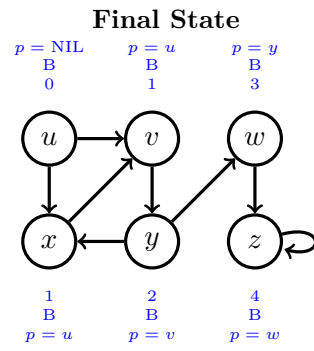
Queue:  $[w]$   
 Current:  $y$   
 Neighbors:  $x, w$



Queue:  $[z]$   
 Current:  $w$   
 Neighbors:  $z$



Queue:  $[\ ]$   
 Current:  $z$   
 Neighbors:  $z$



Queue:  $[\ ]$   
 Current: none  
 Neighbors: none

## 2.2 Pseudocode (BFS)

BFS of graph  $G$  from source  $s$

```

BFS( $G, s$ )
  for each  $u$  in  $V[G]$ 
     $color[u] \leftarrow WHITE$ 
     $p[u] \leftarrow NIL$ 
     $d[u] \leftarrow infinity$ 

   $color[s] \leftarrow GRAY$ 
   $d[s] \leftarrow 0$ 
  initialize empty queue  $Q$ 
  ENQUEUE( $Q, s$ )

  while  $Q$  is not empty
     $u \leftarrow DEQUEUE(Q)$ 
    for each  $v$  in  $Adj[u]$ 
      if  $color[v] = WHITE$ 
         $color[v] \leftarrow GRAY$ 
         $d[v] \leftarrow d[u] + 1$ 
         $p[v] \leftarrow u$ 
        ENQUEUE( $Q, v$ )
     $color[u] \leftarrow BLACK$ 

```

## 2.3 Time complexity (BFS)

With an adjacency-list representation:

- Each vertex is enqueued/dequeued ( $\mathcal{O}(1)$  for each) at most once  $\Rightarrow \mathcal{O}(V)$
- Scanning all adjacency lists for neighbors over the whole run  $\Rightarrow \mathcal{O}(E)$

Therefore, BFS runs in  $\mathcal{O}(V+E)$ .

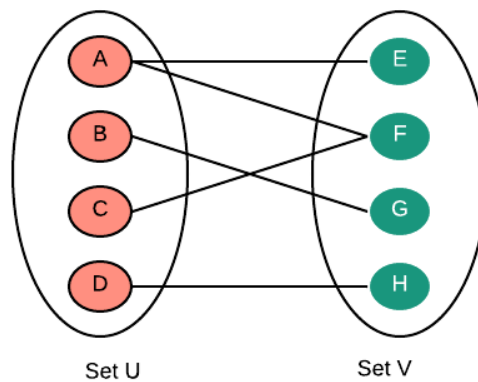
## 2.4 Applications of BFS

### 2.4.1 Shortest Path using BFS

BFS gives the minimum number of edges from  $s$  to each reachable vertex via  $d[v]$  and the predecessor chain  $\pi[v]$ . It works only when  $G$  is unweighted or equally-weighted where minimum distance means minimum number of edges. BFS does not consider cost/weights of edges.

### 2.4.2 Bipartite Graph Check using BFS

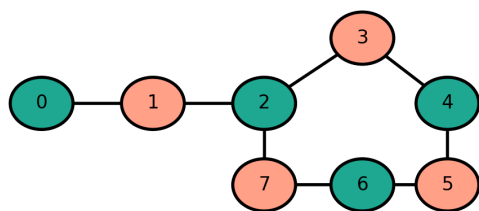
BFS can check if vertices of a graph  $G$  can be divided into two disjoint sets such that all edges connect a vertex in one set to a vertex in another and there are no edges between vertices in the disjoint sets. Equivalently, the graph is bipartite if the graph can be colored with **two colors** so that no adjacent vertices share the same color.



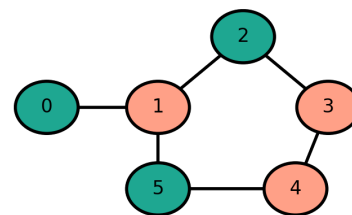
Disjoint Sets of a Bipartite Graph

A graph  $G$  is bipartite if it can be colored using two colors such that no adjacent nodes have the same color.

- Any linear graph with no cycle is always a bipartite graph
- With a cycle, any graph with an even cycle length is also a bipartite graph



Bipartite



Non-Bipartite

Detection of Bipartite and Non-Bipartite using alternating colors

Bipartite-BFS of graph  $G$  from source  $s$

```

BIPARTITE-BFS( $G, s$ )
  for each  $u$  in  $V[G]$ 
     $color[u] \leftarrow UNCOLORED$ 

   $color[s] \leftarrow RED$ 
  initialize empty queue  $Q$ 
  ENQUEUE( $Q, s$ )

  while  $Q$  is not empty
     $u \leftarrow DEQUEUE(Q)$ 
    for each  $v$  in  $Adj[u]$ 
      if  $color[v] = UNCOLORED$ 
         $color[v] \leftarrow opposite(color[u])$ 
        ENQUEUE( $Q, v$ )
      else if  $color[v] = color[u]$ 
        return Not Bipartite

  return Bipartite

```

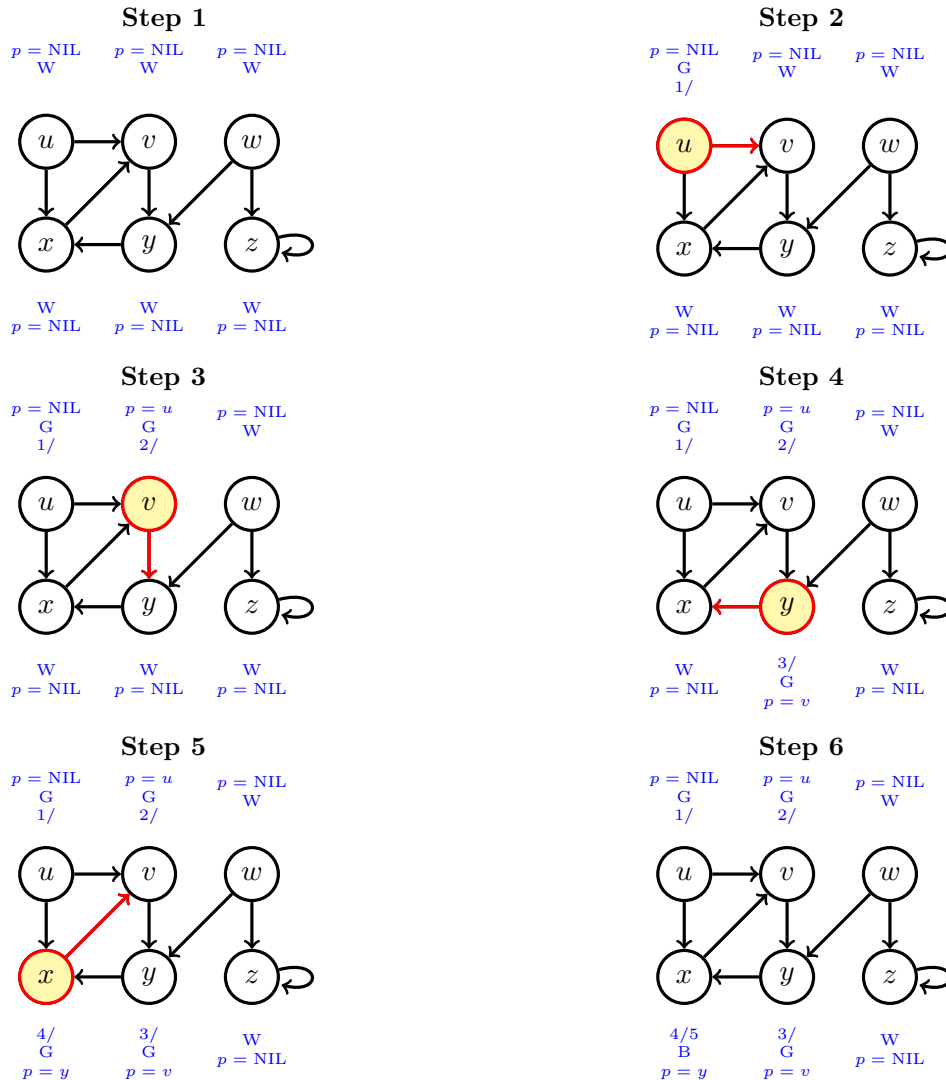
### 3 Depth-First Search (DFS)

Depth-First Search (DFS) explores edges out of the most recently discovered vertex  $v$  that still has unexplored edges leaving it. Once all of  $v$ 's edges have been explored, the search backtracks to explore edges leaving the vertex from which  $v$  was discovered. This process continues until all vertices that are reachable from the original source vertex have been discovered. If any undiscovered vertices remain, then DFS selects one of them as a new source, repeating the search from that source. The algorithm repeats this entire process until it has discovered every vertex.

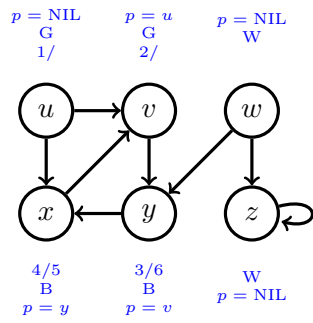
DFS also keeps track of:

- $\text{color}[v]$  : state of each vertex
  - White (W): undiscovered
  - Gray (G): discovered but not finished processing
  - Black (B): processed
- $\pi[v]$  or  $p[v]$  : predecessor (parent) of  $v$
- $d[v]$  : discovery time
- $f[v]$  : finish time

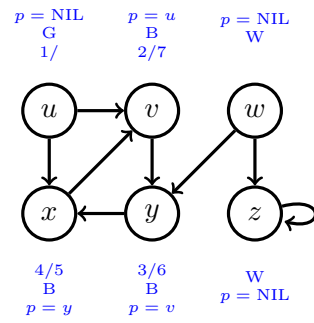
#### 3.1 Simulation (DFS from source $u$ )



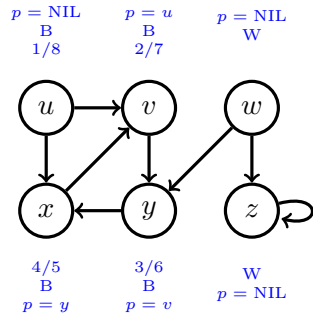
Step 7



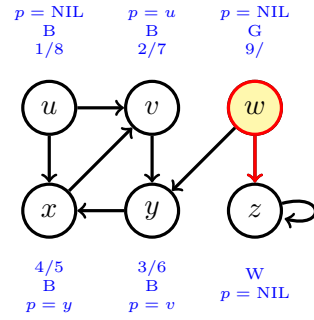
Step 8



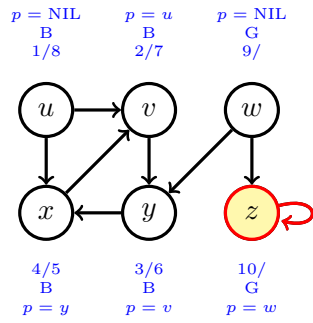
Step 9



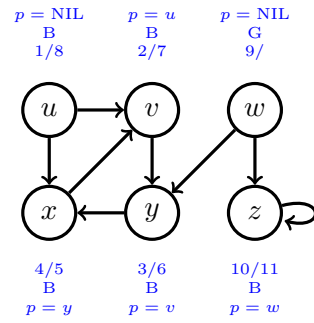
Step 10



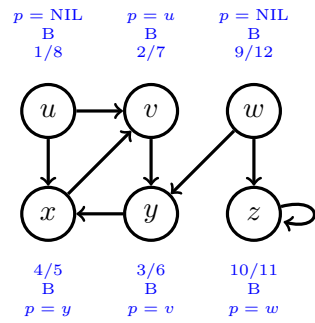
Step 11



Step 12



Final State



### 3.2 Pseudocode (DFS)

#### DFS of graph $G$

```

DFS(G)
  for each  $u$  in  $V[G]$ 
    color[u]  $\leftarrow$  WHITE
    p[u]  $\leftarrow$  NIL

  time  $\leftarrow$  0
  for each  $u$  in  $V[G]$ 
    if color[u] = WHITE
      DFS-VISIT(G, u)

DFS-VISIT(G, u)
  color[u]  $\leftarrow$  GRAY
  time  $\leftarrow$  time + 1
  d[u]  $\leftarrow$  time
  for each  $v$  in Adj[u]
    if color[v] = WHITE
      p[v]  $\leftarrow$  u
      DFS-VISIT(G, v)
  color[u]  $\leftarrow$  BLACK
  time  $\leftarrow$  time + 1
  f[u]  $\leftarrow$  time

```

### 3.3 Time complexity (DFS)

With an adjacency-list representation:

- Each vertex is discovered and processed at most once by DFS-VISIT  $\Rightarrow \mathcal{O}(V)$
- Scanning all adjacency lists for neighbors over the whole run  $\Rightarrow \mathcal{O}(E)$

Therefore, DFS runs in  $\mathcal{O}(V + E)$ .

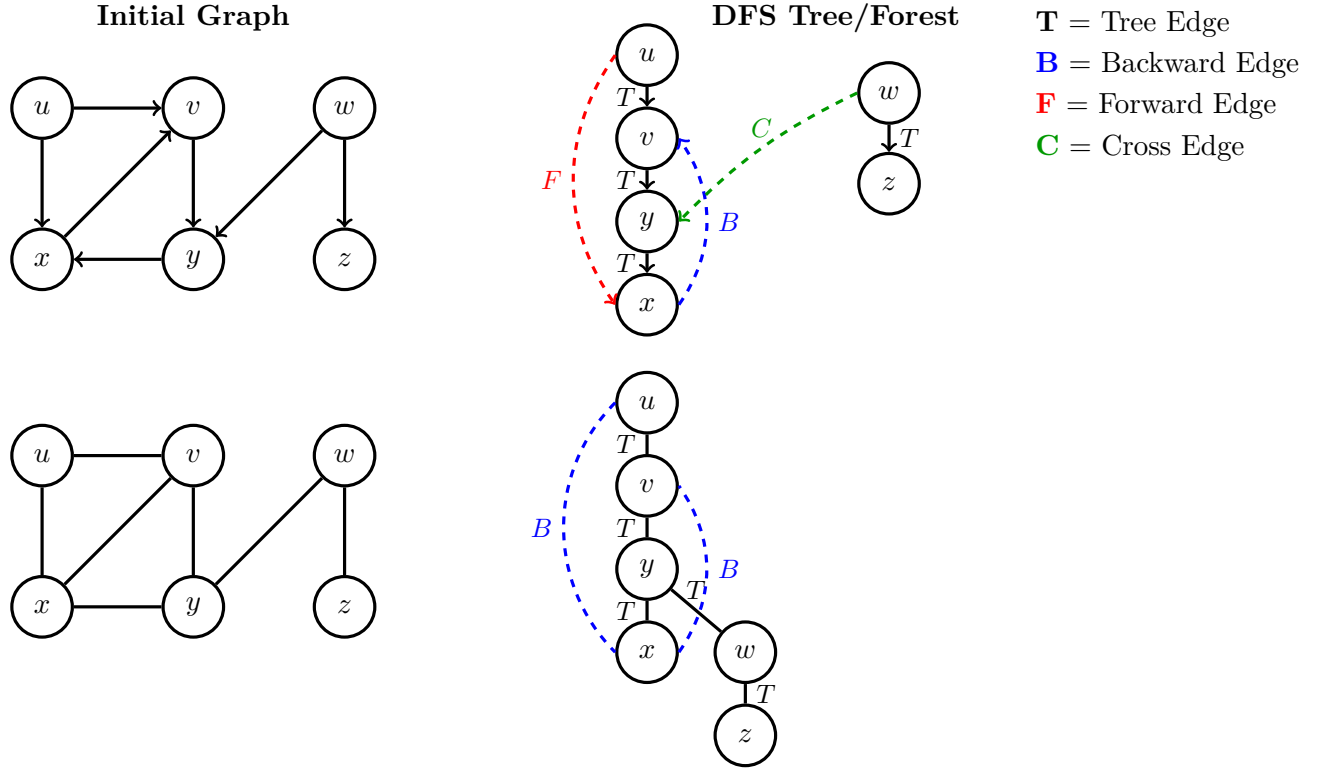
### 3.4 Applications of DFS

#### 3.4.1 Edge classification using DFS

After DFS on a directed graph, edges can be classified into four types:

- **Tree edge:** discovered during DFS traversal
- **Back edge:** points to an ancestor in the DFS tree
- **Forward edge:** points to a descendant in the DFS tree
- **Cross edge:** connects vertices across different branches/subtrees

However, in an **undirected graph**, every edge is either a **tree edge** or a **back edge**.



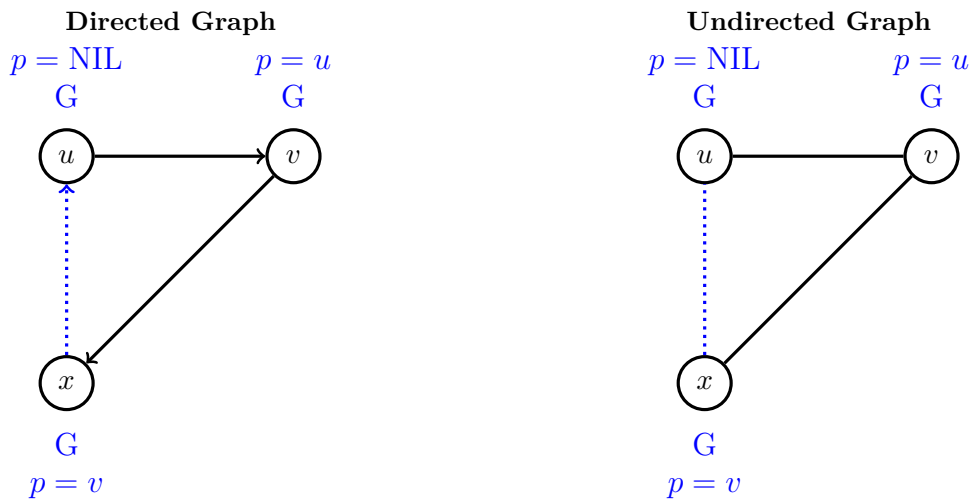
For an edge  $(u \rightarrow v)$ , discovery/finish times help classify edges:

- Tree/Forward:  $u.d < v.d < v.f < u.f$
- Back:  $v.d < u.d < u.f < v.f$
- Cross:  $v.d < v.f < u.d < u.f$

### 3.4.2 Cycle detection using DFS

Using white-gray-black coloring, a directed graph contains a cycle if DFS encounters an edge from the current gray vertex to another **gray** vertex (a back edge to an ancestor).

However, white-gray-black coloring alone is not enough for undirected graphs because every edge appears in both directions. While doing DFS, we must also track the **parent** of each vertex. If we see a visited neighbor that is *not* the parent, then a cycle exists.





## Cycle Detection using DFS

```
HAS-CYCLE-UNDIRECTED(G)
  for each u in V[G]
    color[u] ← WHITE
    parent[u] ← NIL

  for each u in V[G]
    if color[u] = WHITE
      if DFS-CYCLE-UNDIRECTED(G, u, parent[u]) = true
        return Cycle Exists

  return No Cycle

DFS-CYCLE-UNDIRECTED(G, u, parent)
  color[u] ← GRAY

  for each v in Adj[u]
    if color[v] = WHITE
      parent[v] ← u
      if DFS-CYCLE-UNDIRECTED(G, v, u) = true
        return true
    else if color[v] = GRAY and v != parent
      return true

  color[u] ← BLACK
  return false
```